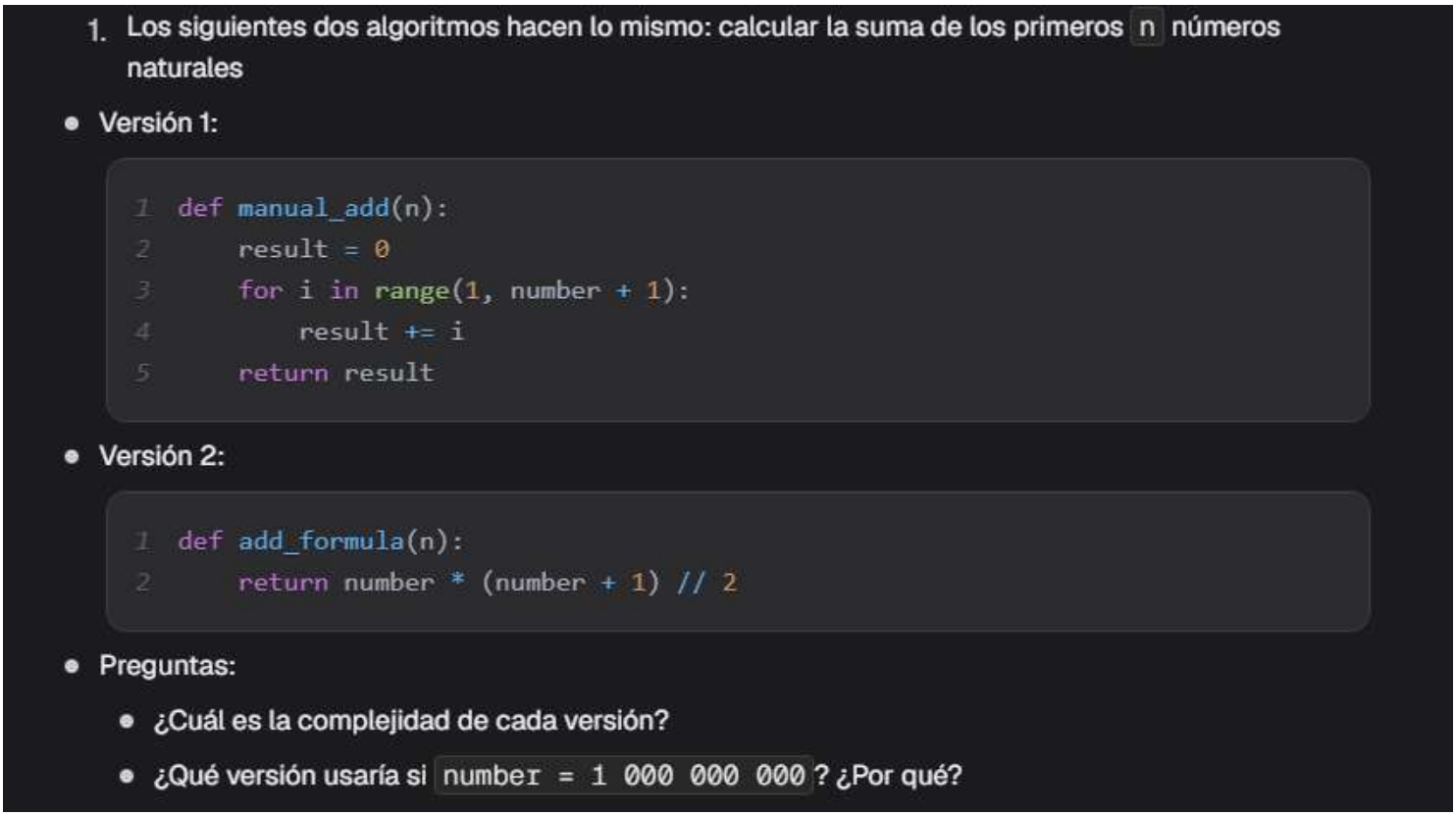
Big O notation extra



1. Los siguientes dos algoritmos hacen lo mismo: calcular la suma de los primeros `n` números naturales

- Versión 1:

```python
def manual_add(n):
    result = 0
    for i in range(1, number + 1):
        result += i
    return result
```

- Versión 2:

```python
def add_formula(n):
    return number * (number + 1) // 2
```

- Preguntas:
  - ¿Cuál es la complejidad de cada versión?
  - ¿Qué versión usaría si `number = 1 000 000 000`? ¿Por qué?

Primero, es importante aclarar que ambos códigos tienen un error cuando cambian n por number de una línea a otra sin ningún tipo de asignación, pero asumo que es n siempre.

Para la versión 1 tenemos un ciclo cuyo número de iteraciones depende del valor de n, por lo tanto el tiempo es O(n). Por otro lado, la versión 2 que utiliza la fórmula para calcular el valor de la suma de la serie tiene un tiempo constante O(1).

Para usar un millón como entrada usaría, la versión 2 porque la versión 1 me haría iterar un millón de veces, mientras que la versión 2 simplemente ejecuta 1 suma, como dice la notación Big O, su tiempo de ejecución es siempre constante.

Big O notation extra

1. Considere los siguientes dos algoritmos:

```python
def linear_search(my_list, target):
    for item in my_list:
        if item == target:
            return True
    return False
```

```python
def binary_search(my_list, target):
    low = 0
    high = len(lst) - 1
    while low <= high:
        mid = (low + high) // 2
        if my_list[mid] == target:
            return True
        elif my_list[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return False
```

- Preguntas:
  - ¿Cuál es la complejidad de cada algoritmo?
  - ¿En qué condiciones conviene usar cada uno?
  - ¿Qué pasa si la lista no está ordenada?

- Para linear_search tenemos un loop que se ejecutaría tantas veces como elementos tenga la lista de entrada, lo cual es equivalente a un tiempo lineal O(n). Por otra parte, binary_search divide la lista a la mitad en cada iteración, de esta forma, se reduce la cantidad de iteraciones máximas a la mitad también, como vimos esto es equivalente a un tiempo logarítmico O(log n).
- La búsqueda lineal es útil cuando la lista es pequeña, incluso si esta desordenada. Mientras que la búsqueda binaria es ideal para listas grandes ordenadas debido a que aplica el principio de dividir para vencer.
- Como dijimos en el punto anterior, solo la búsqueda lineal era resiliente a que la lista careciera de orden, mientras que la búsqueda binaria depende de que la lista este ordenada para funcionar.

Big O notation extra

1. Analice la siguiente función:

```
def print_all_pairs(my_dict):
    for key1 in my_dict:
        for key2 in my_dict:
            print(f"{key1}-{key2}")
```

- Preguntas:
  - ¿Cuál es la complejidad temporal?
  - ¿Cuanto dura si hay `1` millón de claves?

- El tiempo de ejecución es cuadrático O(n^2).
- Si hay 1 millón de claves entonces durará (1 000 000)^2 segundos, dicho de otra forma, 1 billón de segundos